

DOCUMENTO DE DISEÑO DE SOFTWARE (SDD)

CraftForge Web - Administrador de Modpacks Personal Estilo CurseForge

Proyecto:	CraftForge Web (Instancia Auto-alojada)	Fecha:	Julio 2026
Autor:	Desarrollo Interno / Personal	Versión:	v1.0.0
Entorno Destino:	Docker / Local / Coolify (Ubuntu Server)	Usuarios Objetivos:	1 - 3 Usuarios Concurrentes

1. Introducción y Objetivos del Sistema

El presente documento detalla las especificaciones de diseño, arquitectura e interfaz para **CraftForge Web**, una aplicación web de uso personal diseñada para simplificar la creación, gestión y exportación de modpacks de Minecraft de manera idéntica a la experiencia de la aplicación de escritorio de CurseForge, pero operando completamente desde el navegador.

Objetivos clave:

- **Simplicidad de Despliegue:** Todo el ecosistema debe ser orquestado mediante un único archivo de Docker Compose, facilitando las pruebas locales y permitiendo el despliegue directo en paneles PaaS como Coolify.
- **Optimización de Recursos:** Al estar acotado a un entorno personal (1-3 usuarios simulados), el backend y la base de datos se dimensionan de forma compacta, priorizando el uso eficiente del almacenamiento en disco a través de la recolección automática de archivos temporales.
- **Fidelidad Visual:** La interfaz de usuario imitará la distribución y paleta de colores oscura clásica de CurseForge para proveer un entorno familiar e intuitivo al crear perfiles de juego.

2. Arquitectura del Sistema e Infraestructura Docker

La aplicación se estructura como un sistema multi-contenedor aislado en una red interna privada. Esto garantiza la independencia de los servicios y emula de forma idéntica un entorno de producción, incluso corriendo en una máquina local.

Servidor / Contenedor	Tecnología Recomendada	Función Principal
frontend	Next.js / Nuxt.js (Node)	Renderiza la interfaz web de usuario. Ejecuta Server-Side Rendering (SSR) para acelerar la carga visual.
backend-api	Node.js (TS) / Go	Procesa peticiones HTTP del front, valida credenciales, expone endpoints proxy hacia CurseForge.
queue-worker	Mismo entorno Backend	Proceso dedicado de fondo encargado puramente de la descarga paralela de archivos .jar y compresión de ZIPs.
database	PostgreSQL (Alpine)	Almacenamiento persistente de perfiles, metadatos de los mods seleccionados y estados del sistema.
cache-broker	Redis (Alpine)	Cachea las respuestas de la API externa (24h de expiración) y gestiona la cola de mensajería asíncrona.

2.1 Configuración de Orquestación: docker-compose.yml

A continuación se detalla la configuración del archivo de orquestación estructurado para ser desplegado instantáneamente en entornos locales o importado en Coolify:

```

version: '3.8'

services:
  database:
    image: postgres:15-alpine
    container_name: craftforge-db
    environment:
      POSTGRES_USER: craftforge_admin
      POSTGRES_PASSWORD: password_seguro_local
      POSTGRES_DB: craftforge_db
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - craftforge-net

  cache-broker:
    image: redis:7-alpine
    container_name: craftforge-cache
    networks:
      - craftforge-net

  backend-api:
    image: craftforge-backend:latest
    container_name: craftforge-api
    environment:
      DATABASE_URL: postgres://craftforge_admin:password_seguro_local@database:5432/
craftforge_db
    REDIS_URL: redis://cache-broker:6379
    CURSEFORGE_API_KEY: ${CURSEFORGE_API_KEY}
    PORT: 3000
    volumes:
      - shared-scratch:/app/scratch
    depends_on:
      - database
      - cache-broker
    networks:
      - craftforge-net

  queue-worker:
    image: craftforge-backend:latest
    container_name: craftforge-worker
    command: ["npm", "run", "start:worker"]
    environment:
      DATABASE_URL: postgres://craftforge_admin:password_seguro_local@database:5432/
craftforge_db
    REDIS_URL: redis://cache-broker:6379
    volumes:
      - shared-scratch:/app/scratch
    depends_on:

```

```

    - cache-broker
networks:
    - craftforge-net

frontend:
    image: craftforge-frontend:latest
    container_name: craftforge-ui
    ports:
        - "8080:3000"
    environment:
        NEXT_PUBLIC_API_URL: http://localhost:3000
    depends_on:
        - backend-api
    networks:
        - craftforge-net

volumes:
    db-data:
    shared-scratch:

networks:
    craftforge-net:
        driver: bridge

```

Nota de Despliegue en Coolify: Al importar este archivo en Coolify, las variables de entorno como \$ {CURSEFORGE_API_KEY} pueden configurarse directamente desde la interfaz gráfica del panel, mapeando el puerto 8080 del contenedor de frontend al dominio público asignado.

3. Diseño de la Interfaz de Usuario (UI/UX Estilo CurseForge)

Para garantizar la fidelidad con la aplicación de escritorio nativa de CurseForge, el diseño web adopta una estructura de tres bloques principales bajo un esquema cromático oscuro (Dark Theme).

3.1 Paleta de Colores y Estilos Base

- **Fondo de la Aplicación:** Negro Grisáceo Profundo (#141416)
- **Contenedores y Tarjetas:** Gris Carbón (#1c1e22)
- **Color de Acento / Botones de Acción:** Naranja CurseForge (#e05320 o #ff8000)
- **Bordes y Separadores:** Gris Oscuro Línea (#2d3139)
- **Tipografía Primaria:** Arial / Helvetica Neue, color blanco (#ffffff) y gris atenuado (#a0a5b0).

3.2 Estructura del Layout (Distribución en Navegador)

La pantalla se divide utilizando un diseño de tabla de bloques de la siguiente manera:



4. Modelo de Datos (Esquema de Base de Datos relacional)

Para soportar las banderas de entorno de cliente/servidor, los perfiles independientes y el árbol de dependencias, se establece la siguiente estructura en PostgreSQL:

Tabla: modpacks

Almacena la cabecera del perfil creado por el usuario.

Campo	Tipo de Datos	Descripción
id	SERIAL (PK)	Identificador único del modpack.
nombre	VARCHAR(150)	Nombre asignado por el usuario.
version_minecraft	VARCHAR(20)	Ej: "1.20.1", "1.12.2".
modloader_tipo	VARCHAR(30)	Forge, NeoForge, Fabric o Quilt.
modloader_version	VARCHAR(50)	Versión exacta del cargador de mods elegido.

Tabla: modpack_mods

Contiene la relación de mods agregados a cada modpack, gestionando dependencias y asignaciones de entorno.

Campo	Tipo de Datos	Descripción
id	SERIAL (PK)	Identificador de la relación.
modpack_id	INT (FK)	Enlace con la tabla modpacks. Al borrar el modpack se borra en cascada.
curseforge_project_id	INT	ID único del proyecto entregado por la API oficial.
curseforge_file_id	INT	ID específico del archivo .jar seleccionado para esa versión de Minecraft.
nombre_mod	VARCHAR(200)	Caché de texto del nombre del mod para evitar consultas repetitivas a la API.
entorno_destino	VARCHAR(15)	Valores válidos: BOTH, CLIENT_ONLY, SERVER_ONLY.
es_dependencia	BOOLEAN	TRUE si el mod se autoinstaló por requerimiento algorítmico; FALSE si fue por el usuario.
padre_proyecto_id	INT (Nullable)	Registra qué ID de mod solicitó esta dependencia, permitiendo bloqueos en la interfaz de usuario.

5. Lógica de Negocio y Flujo de Procesamiento Algorítmico

5.1 Algoritmo de Resolución de Dependencias Recursivo

Cuando el usuario realiza una solicitud en la API para instalar un mod, el backend ejecuta la siguiente lógica lógica:

- **Paso 1:** Verifica en la caché de Redis si las dependencias del `project_id` para la versión dada de Minecraft ya existen. Si existen, salta al Paso 3.
- **Paso 2:** Si no están en caché, consulta el endpoint de CurseForge `/v1/mods/{modId}/files/{fileId}`. Extrae el arreglo de dependencias (`dependencies`) filtrando las que tengan `relationType: 3` (Required Dependency). Almacena el resultado en Redis por 24 horas.
- **Paso 3:** Para cada dependencia requerida hallada, el motor invoca recursivamente este algoritmo.
- **Paso 4:** Una vez mapeado todo el árbol libre de duplicados, guarda cada registro en la tabla `modpack_mods` marcándolos con `es_dependencia = TRUE` y asignando el `padre_proyecto_id` correspondiente.

5.2 Flujo de Compilación y Limpieza de ZIPs Completos

Para evitar bloqueos HTTP causados por descargas masivas de archivos ejecutables pesados, el proceso de exportación directa se delega asíncronamente al contenedor de Worker:

- **Encolamiento:** La API recibe la solicitud de exportación (Ej: Servidor) y empuja un mensaje estructurado a la cola de Redis administrada por BullMQ / Celery. La API responde inmediatamente con un código HTTP 202 (Accepted).
- **Descarga en Scratch:** El queue-worker toma el mensaje, lee la base de datos y crea un directorio temporal único en el volumen compartido (/app/scratch/{task_id}).
- **Filtrado Discriminatorio:** Descarga secuencial o paralelamente (máximo 3 descargas simultáneas para mitigar bloqueos de IP) los archivos .jar. Si la solicitud fue para *Servidor*, el Worker omite los registros parametrizados en base de datos como CLIENT_ONLY.
- **Compresión:** El Worker genera la estructura de carpetas estándar (/mods), copia los archivos de configuraciones correspondientes y comprime todo en un archivo único de descarga.
- **Recolección de Basura:** Mediante un demonio Cron que se ejecuta en segundo plano en el backend, los archivos binarios contenidos dentro del volumen shared-scratch que tengan marcas de tiempo superiores a 120 minutos son purgados de forma permanente, garantizando que el almacenamiento del VPS local nunca se sature.

6. Conclusión de Diseño

Este diseño garantiza un sistema ágil, fácil de mantener en un entorno casero mediante Docker y listo para crecer si se requiere. La interfaz emula los puntos fuertes de CurseForge, dándole al usuario un control total sobre qué archivos se van al cliente de juego y cuáles al servidor de Minecraft independiente sin configuraciones manuales propensas a errores.